



UNIVERSIDAD DE CARABOBO
FACULTAD EXPERIMENTAL DE CIENCIAS
Y TECNOLOGÍA
DEPARTAMENTO DE COMPUTACIÓN
ARQUITECTURA DEL COMPUTADOR



PRÁCTICA DE LABORATORIO #3

Manejo de Dispositivos de Entrada/Salida Mapeados en Memoria (Memory-Mapped I/O) en Ensamblador MIPS32

Estudiantes:

Nathaly Campos *C.I. 32.460.314*

David Serrano *C.I. 31.000.672*

Profesor:

José Canache

1. **Explique cómo se organiza la memoria cuando un sistema utiliza memory-mapped I/O. ¿En qué región de memoria se suelen mapear los dispositivos? ¿Qué implicaciones tiene para las instrucciones `lw` y `sw`?**

En un sistema que utiliza *Memory-Mapped I/O* (E/S Mapeada en Memoria), el mapa de direcciones unificado comparte el espacio de memoria RAM tradicional con los registros de control, estado y datos de los periféricos hardware.

- **Región de memoria:** En la arquitectura MIPS32, los dispositivos suelen mapearse en el segmento superior del mapa de direcciones virtuales, específicamente dentro del espacio reservado para el kernel/sistema (`0xFFFF0000` - `0xFFFFFFFF`).
- **Implicaciones para `lw` y `sw`:** No se requieren instrucciones especializadas de entrada/salida. Las instrucciones estándar de carga (`lw`) y almacenamiento (`sw`) se reutilizan para interactuar con el periférico. Una instrucción `sw` en la dirección del registro de control inicia un evento de hardware, mientras que una instrucción `lw` lee el estado o los datos del dispositivo.

2. **¿Cuál es la principal diferencia entre memory-mapped I/O y la entrada/salida por puertos? ¿Qué ventajas y desventajas tiene cada enfoque? ¿Por qué MIPS32 utiliza principalmente memory-mapped I/O?**

- **Diferencia Principal:** *Memory-Mapped I/O* utiliza el mismo espacio de direcciones y bus para la RAM y los periféricos. Por otro lado, la *E/S por puertos* (Port-Mapped I/O) posee un espacio de direcciones aislado y requiere instrucciones dedicadas (como `IN` y `OUT` en x86).
- **Ventajas y Desventajas:**
 - *Memory-Mapped I/O:* ****Ventaja:**** Simplifica el hardware del procesador al no requerir un bus de control extra ni instrucciones exclusivas. Permite usar todos los modos de direccionamiento sobre los periféricos. ****Desventaja:**** Consume parte del espacio de direccionamiento de memoria RAM principal.
 - *Port-Mapped I/O:* ****Ventaja:**** No reduce el espacio de direcciones de la memoria principal. ****Desventaja:**** Requiere instrucciones adicionales y circuitos de decodificación más complejos en la CPU.
- **Uso en MIPS32:** MIPS32 sigue la filosofía RISC de mantener un conjunto de instrucciones simple y homogéneo. Utilizar *Memory-Mapped I/O* evita la adición de instrucciones de E/S al ISA, optimizando el diseño del camino de datos.

3. En un sistema con memory-mapped I/O: ¿Qué problemas pueden surgir si dos dispositivos usan direcciones solapadas? ¿Cómo se evita este conflicto?

Si dos dispositivos comparten o solapan sus direcciones de memoria, ocurre una ****colisión en el bus de datos****. Durante una lectura (**lw**), ambos periféricos intentarían colocar sus datos simultáneamente en el bus, generando estados indeterminados o cortocircuitos lógicos. En operaciones de escritura (**sw**), ambos dispositivos responderían de forma no deseada.

Solución: Se evita mediante una unidad lógica de ****Decodificación de Direcciones**** (*Address Decoder*) que habilita mediante la señal *Chip Select* (CS) a un único dispositivo a la vez, garantizando que cada registro o periférico ocupe un rango estrictamente disjunto y único en el espacio de memoria.

4. ¿Por qué se considera que el memory-mapped I/O simplifica el diseño del conjunto de instrucciones de un procesador? ¿Qué tipo de instrucciones adicionales serían necesarias si se usara E/S por puertos?

Simplifica el diseño porque elimina la necesidad de crear, decodificar y ejecutar instrucciones especiales para periféricos. Las instrucciones existentes de transferencia de memoria (**lw**, **sw**, **lbu**, **sb**) asumen toda la interacción hardware.

Si se empleara E/S por puertos, la arquitectura necesitaría agregar instrucciones exclusivas como:

- **IN \$rt, port:** Para leer el contenido de un puerto de entrada.
- **OUT port, \$rs:** Para escribir un valor en un puerto de salida.
- Instrucciones de control de estado del bus de puertos especializadas.

5. ¿Qué ocurre a nivel del bus de datos y direcciones cuando el procesador accede a una dirección de memoria que corresponde a un dispositivo? ¿Cómo sabe el hardware que debe acceder a un periférico en lugar de la RAM?

Cuando la CPU ejecuta una instrucción **lw** o **sw** sobre la dirección de un dispositivo:

1. El procesador coloca la dirección física en el ****Bus de Direcciones**** y activa las líneas del ****Bus de Control**** (**MemRead** o **MemWrite**).
2. El circuito decodificador de direcciones del sistema analiza los bits de mayor peso

(MSB) de la dirección.

3. Si la dirección cae en el rango de la RAM, el decodificador activa la memoria principal. Si cae en el rango reservado para E/S, el decodificador desactiva la RAM y activa la línea *Enable/Chip Select* del módulo del periférico correspondiente.
4. El periférico lee o coloca los datos directamente en el ****Bus de Datos****.

6. ¿Es posible que un programa normal (sin privilegios) acceda a un dispositivo mapeado en memoria? ¿Qué mecanismos de protección existen para evitar accesos no autorizados?

En un sistema operativo moderno, ****no es posible**** de forma directa. Los periféricos suelen estar mapeados en regiones del espacio de direcciones reservadas para el modo Kernel (como `kseg0` o `kseg1` en MIPS).

Mecanismos de Protección:

- **Modos de Ejecución del Procesador:** Distinción entre Modo Usuario y Modo Kernel/Privilegiado.
- **Unidad de Manejo de Memoria (MMU):** Utiliza tablas de páginas que configuran bits de protección. Si un proceso en modo usuario intenta ejecutar un `lw` o `sw` sobre una dirección del sistema, la MMU genera una excepción de fallo de protección de memoria (*Memory Access Exception*), transfiriendo el control al SO.

7. ¿Qué técnicas se pueden emplear para evitar esperas activas innecesarias al interactuar con dispositivos?

Para superar la ineficiencia de la espera activa (*Polling*), se utilizan las siguientes alternativas:

- **E/S dirigida por Interrupciones (Interrupt-Driven I/O):** El procesador inicia la operación en el dispositivo y continúa ejecutando otras tareas. Cuando el dispositivo termina la operación o captura un dato, envía una señal de interrupción de hardware a la CPU, obligándola a pausar temporalmente su ejecución para atender la rutina de servicio a interrupción (ISR).
- **Acceso Directo a Memoria (DMA - Direct Memory Access):** Permite que un controlador especial transfiera grandes bloques de datos entre los periféricos y la memoria RAM sin la intervención continua del procesador, liberándolo por completo durante la transferencia.

8. Análisis y Discusión de los Resultados

En esta práctica se implementaron y simularon exitosamente controladores de dispositivos bajo la arquitectura MIPS32 en la plataforma MARS:

- **Sensor de Luminosidad (Ejercicio 1):** Se verificó la lógica de inicialización enviando el valor 0x1 a `LuzControl` e implementando un bucle de comprobación (*polling*) sobre el registro `LuzEstado`. La rutina `LeerLuminosidad` validó correctamente los códigos de estado devueltos (retornando 0 en éxito y -1 ante fallas).
- **Medidor de Tensión Arterial (Ejercicio 2):** La rutina `controlador_tension` demostró el correcto manejo de valores de retorno dobles mediante los registros `$v0` (Presión Sistólica) y `$v1` (Presión Diastólica), asegurando la sincronización hasta que el dispositivo confirmara el término de la medición (`TensionEstado = 1`).
- **Conclusión:** El esquema de *Memory-Mapped I/O* facilita la abstracción y control de periféricos mediante ensamblador MIPS32. Sin embargo, se constató que la técnica de *polling* genera un uso intensivo de CPU, resaltando la importancia del estudio de interrupciones de hardware en arquitecturas reales.